Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04.5 LTS (OrbStack VM, arm64)

Question 2 - Manage Docker containers and images

Question 2: Manage Docker containers and images: list, start, stop, remove containers, tag and push images to a registry, and use Docker Compose for multi-container setups.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64), Docker Engine 29.9.3, Docker Compose v5.7.0. Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

Container lifecycle

```
$ docker run -d --name lab2-web -p 5000:5000 ensp461-lab2:2.0.0
$ docker ps --filter name=lab2-web --format '{{.Names}} {{.Image}} {{.Status}} {{.Ports}}'
lab2-web ensp461-lab2:2.0.0 Up 3 seconds (health: starting) 0.0.0.0:5000->5000/tcp, [::]:5000->
5000/tcp
$ docker stop lab2-web
$ docker inspect lab2-web --format 'exit={{.State.ExitCode}} status={{.State.Status}}'
exit=0 status=exited
$ docker ps -a --filter name=lab2-web --format '{{.Names}} {{.Status}}'
lab2-web Exited (0) Less than a second ago
$ docker start lab2-web
$ docker ps --filter name=lab2-web --format '{{.Names}} {{.Status}}'
lab2-web Up Less than a second (health: starting)
$ docker restart lab2-web
$ curl -s localhost:5000/health
{"status":"UP","version":"2.0.0"}
$ docker rm -f lab2-web
$ docker ps -a --filter name=lab2-web --format '{{.Names}} {{.Status}}'
(empty: container removed)
```

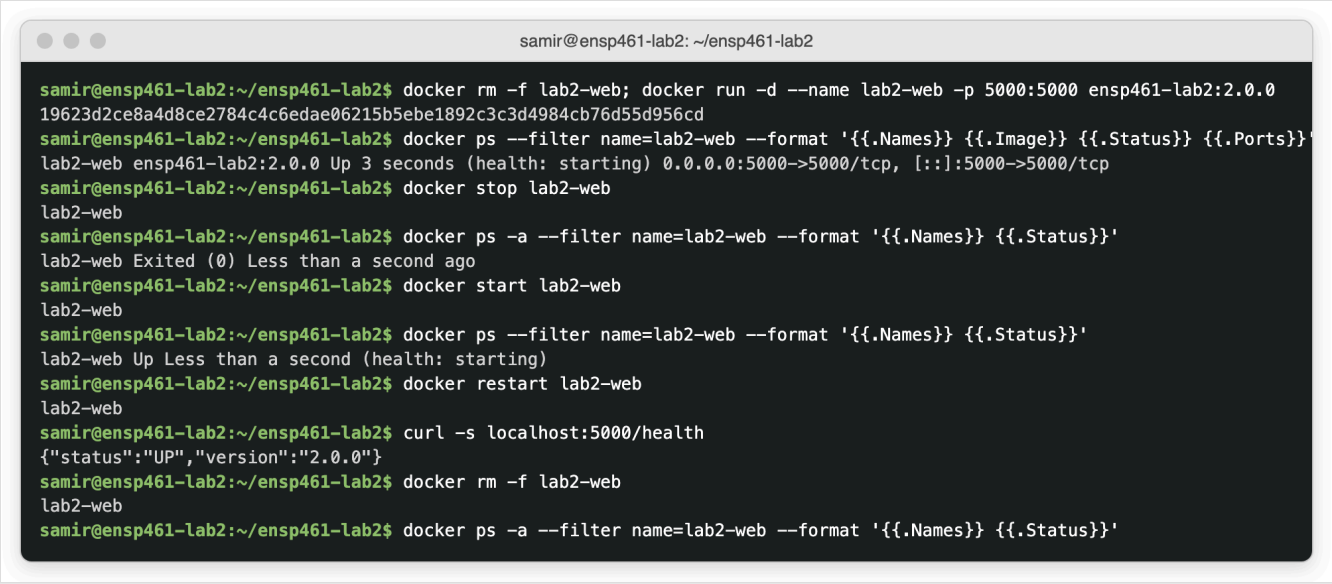


Fig: Container lifecycle: list, stop, start, restart, remove

`docker stop` sends SIGTERM and gunicorn shut down cleanly (exit code 0). A stopped container keeps its writable layer and config, so `docker ps -a` still lists it and `docker start` resumes the same container. Only `docker rm` deletes it.

Tag and push to a registry

The registry is the open-source Distribution registry (`registry:2`) running as a container on port 5001. It is also what the Kubernetes cluster pulls from in Q5 to Q7, so the whole lab runs on one self-hosted registry.

```
$ docker run -d --name lab2-registry --restart unless-stopped -p 5001:5000 public.ecr.aws/docke
r/library/registry:2
$ docker tag ensp461-lab2:2.0.0 localhost:5001/ensp461-lab2:2.0.0
$ docker tag ensp461-lab2:2.0.0 localhost:5001/ensp461-lab2:latest
$ docker push --quiet localhost:5001/ensp461-lab2:2.0.0
$ docker push --quiet localhost:5001/ensp461-lab2:latest
$ curl -s localhost:5001/v2/ensp461-lab2/tags/list
{"name":"ensp461-lab2","tags":["2.0.0","latest"]}
$ docker rmi localhost:5001/ensp461-lab2:2.0.0 localhost:5001/ensp461-lab2:latest
$ docker pull -q localhost:5001/ensp461-lab2:2.0.0
```

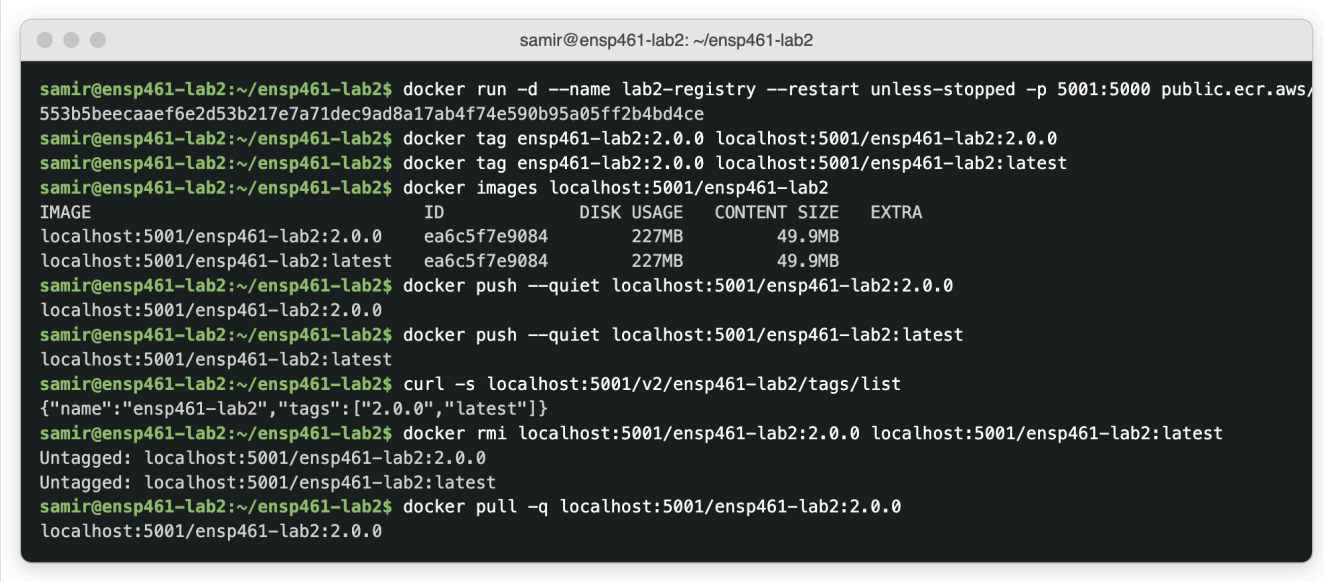


Fig: Tag, push, list tags through the registry API, remove and pull back

Tags are pointers: `2.0.0` and `latest` share image ID `ea6c5f7e9084`, and removing them only untags. The registry HTTP API (`/v2/<name>/tags/list`) confirms both tags were stored, and pulling `2.0.0` back proves the round trip.

Docker Compose (web + Redis)

```
name: lab2
services:
  web:
    build: {context: .., dockerfile: docker/Dockerfile}
    image: ensp461-lab2:2.0.0
    ports: ["5000:5000"]
    environment: {REDIS_HOST: redis, APP_ENV: compose}
    depends_on:
      redis: {condition: service_healthy}
  redis:
    image: public.ecr.aws/docker/library/redis:7-alpine
    volumes: ["redis-data:/data"]
    healthcheck: {test: ["CMD", "redis-cli", "ping"], interval: 5s, timeout: 3s, retries: 5}
volumes:
  redis-data:
```

Step	`/visits`
`up -d --build`, two requests	1, 2
`restart web`	3
`down` then `up -d` (containers recreated, volume kept)	4
`down -v`	volume removed

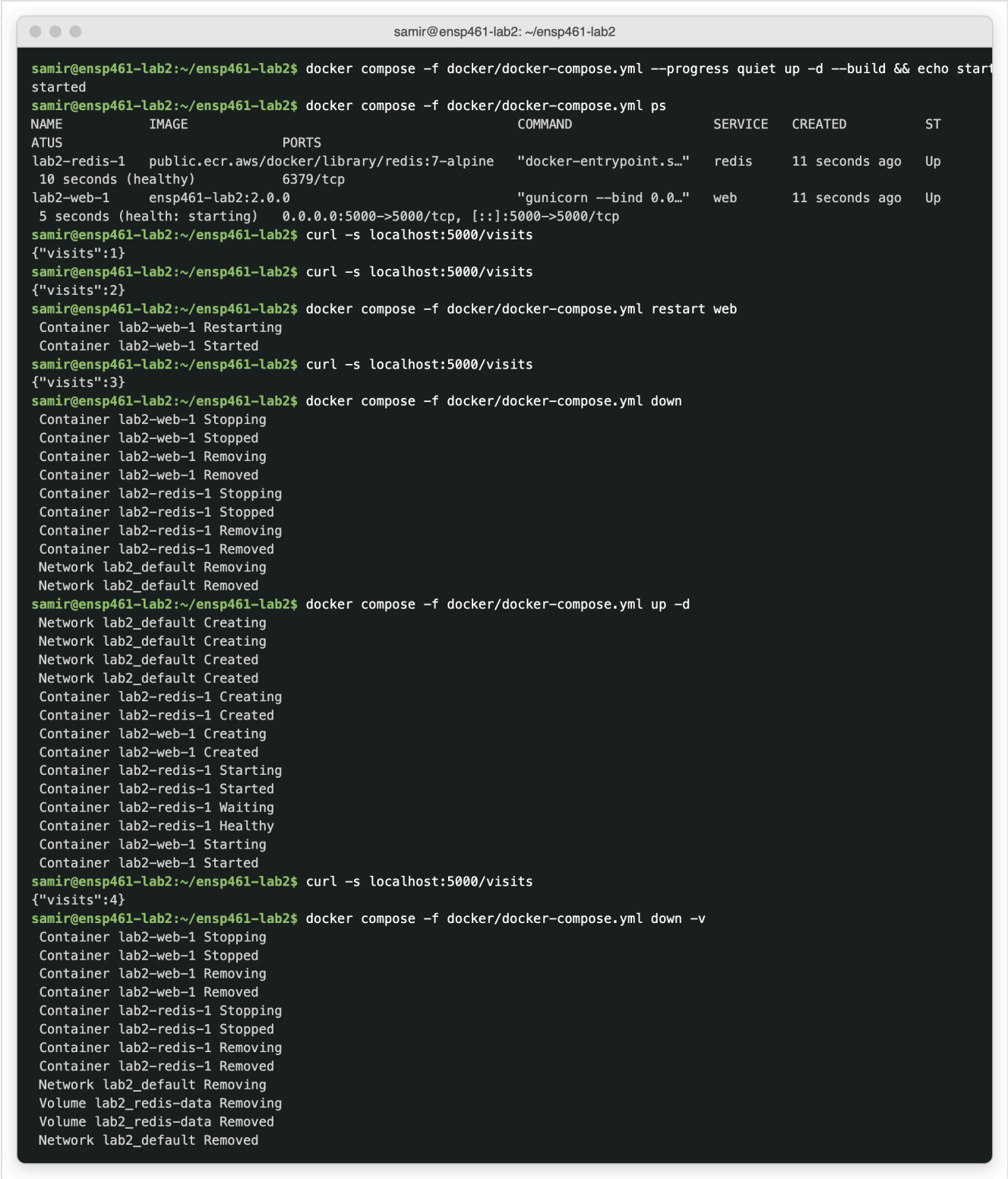


Fig: Compose stack: start, counter, restart, down and up

Observations:

- The web service reaches Redis by the service name `redis` on the project network `lab2_default`.
- `depends_on: condition: service_healthy` held web back until Redis answered `PING` (`lab2-redis-1 Healthy` before `lab2-web-1 Starting`).
- The counter survived both a web restart and a full `down`/`up`, because Redis state lives in the named volume `redis-data`. Only `down -v` deletes it.